Interface

Nyckelordet ”interface” i Java

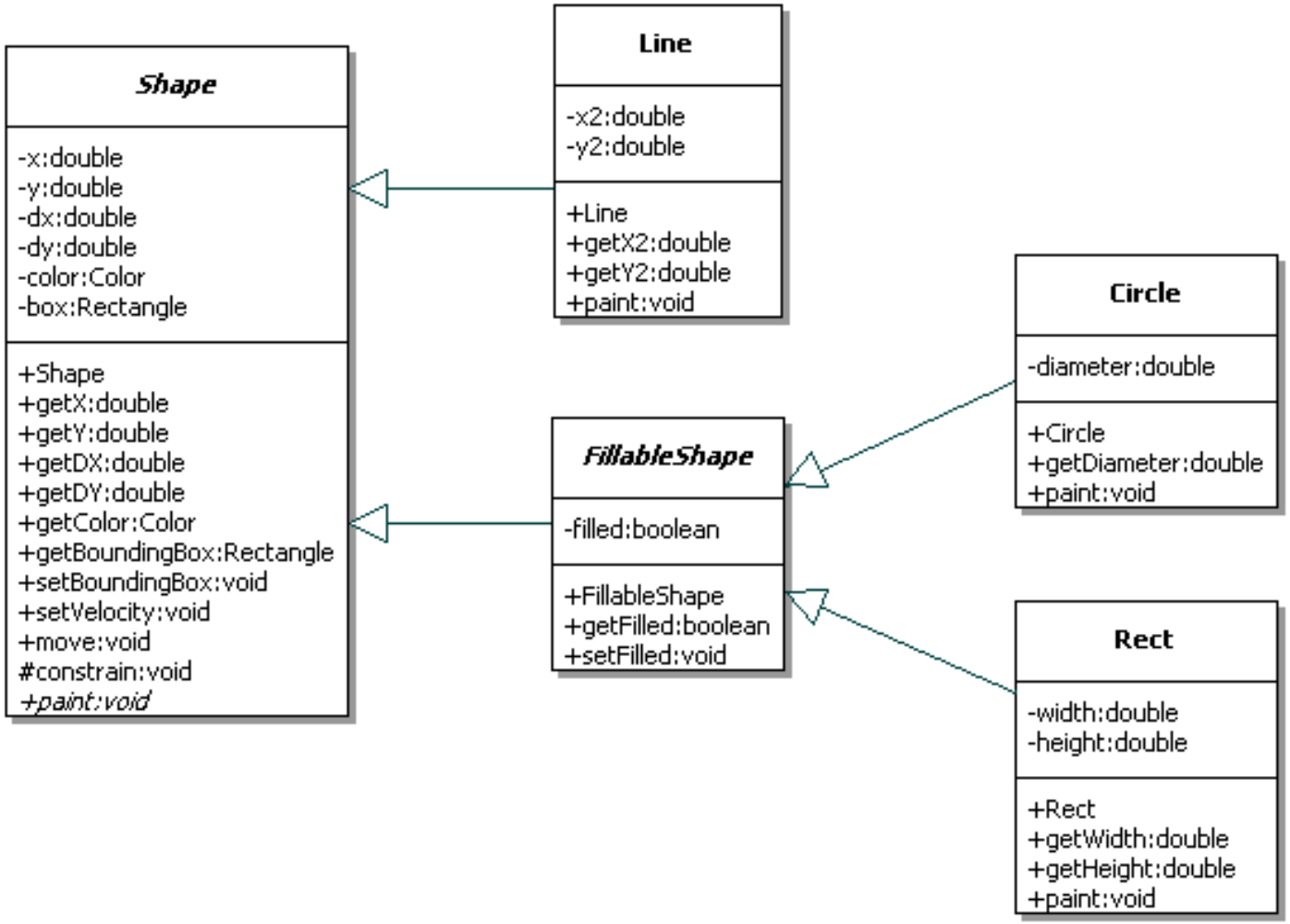
Kanon för klasser i Java

Multipelt arv?

- Skulle innebära att vi ärver beteende, dvs. metoder och datamedlemar, från flera (super-) klasser …
- … vilket skulle innebära att vi får en klass som modellerar flera olika roller
- Problem
  - Klasser ska ha hög kohesion – det är en dålig idé att låta en klass ha multipla roller – varje klass bör modellera en och endast en sak
  - Om vi ärver datamedlemmar eller metoder med samma namn från två olika superklasser – vilken gäller?
- Java tillåter *inte multipelt arv* utan endast arv, extends, från en enda superklass

Java: en klass får endast ärva från *en* superklass

En superklass kan ha flera subklasser, men en subklass får endast ha en enda superklass



Interface?

- Interface = gränssnitt, exempelvis
  - mellan människa och applikation, användargränssnitt
  - mellan objekt/klasser i applikationskod, klassernas publika gränssnitt
- I Java även nyckelordet ”interface” …
- … en uppsättning publika och *abstrakta* metoder
- Inga datamedlemmar och ingen implementation i ett Java-interface
  - förutom klasskonstanter, static final
  - numera tillåtet med ”default-implementation” av metoder
- En klass kan implementera ett eller flera interface (men endast ärva från en superklass)
- Här: interface = ett (del-) gränssnitt för ett objekt gentemot andra objekt
  - Objektet kan ha flera gränssnitt(!)

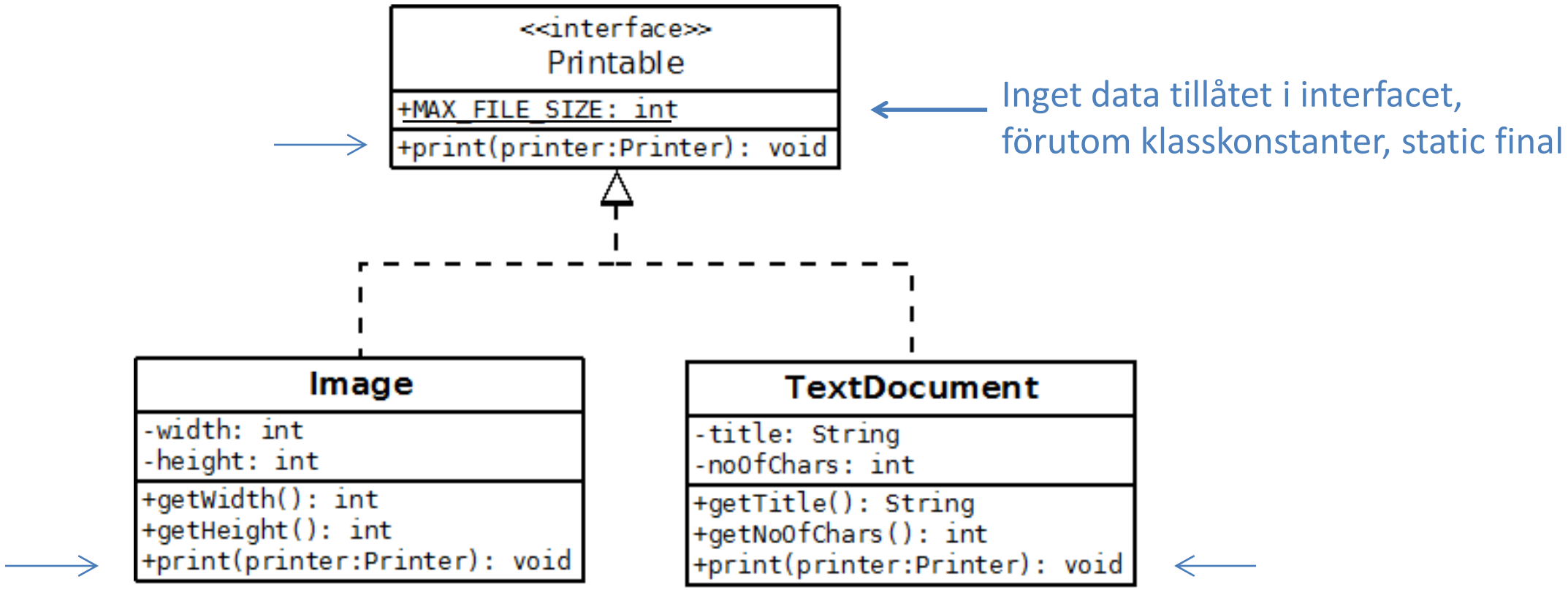
Exempel på ett interface

```
/**
 * A class implementing interface Printable must define
 * how to print the instance on a printer.
 */
public interface Printable {

        abstract public void print(Printer printer);
        // possibly more abstract methods...

        public static final int MAX_FILE_SIZE = 1_000; // MB
}
```

Exempel


<<interface>>
Printable
+MAX_FILE_SIZE: int
+print(printer:Printer): void
Inget data tillåtet i interfacet,
förutom klasskonstanter, static final
Image
-width: int
-height: int
+getWidth(): int
+getHeight(): int
+print(printer:Printer): void
TextDocument
-title: String
-noOfChars: int
+getTitle(): String
+getNoOfChars(): int
+print(printer:Printer): void

Exempel, implementerande klass

```
public class Image implements Printable {

        private int width, height;
        private int[][] pixels;

        public int getWidth() { return width; }
        public int getHeight() { return height; }

        @Override
        public void print(Printer printer) {
                printer.scaleDocument(width, height);
                // Define how to print this image
                // ...
                printer.addToQueue(this);
        }

        // and more Image specific methods ...
}
```

Använda objekt av implementerande typ

```
public class Printer {

    private ArrayList<Printable> printQueue;

    public Printer() {
        printQueue = new ArrayList<>();
    }

    public void addToQueue(Printable job) {
        printQueue.add(job);
    }

    public void printAll() {
        for (Printable p : printQueue) {
            p.print(this);
        }

        printQueue.clear();
    }
}
```

```
public class PrintableMain {

    public static void main(String[] args) {

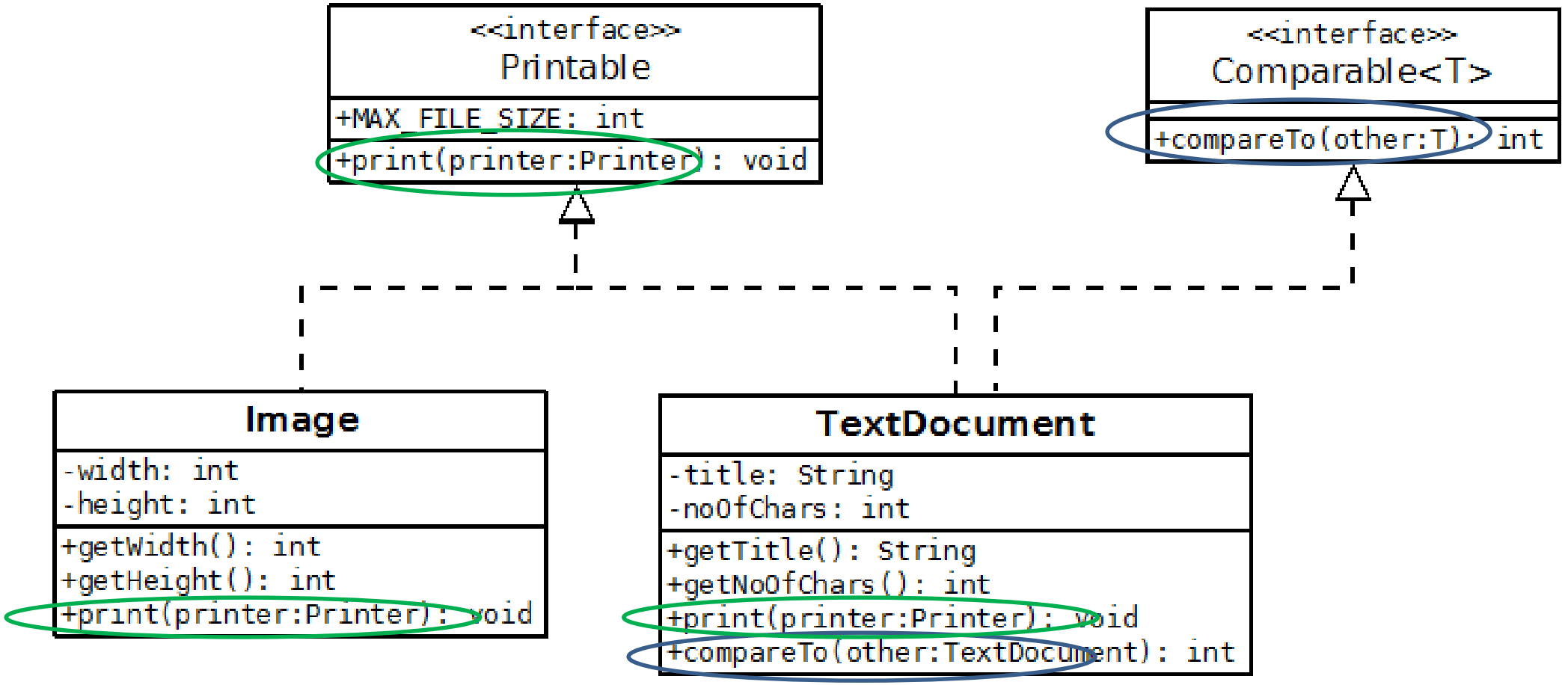
        Printer printer = new Printer();

        TextDocument document = new TextDocument();
        printer.addToQueue(document);

        Image image = new Image();
        printer.addToQueue(image);

        printer.printAll();
    }
}
```

Exempel, klass som implementerar flera interface



Exempel, implementerande klass

```
public class TextDocument implements Printable, Comparable<TextDocument> {
    private String title;
    private LocalDate lastUpdated;
    // …
    public String getTitle() { return title; }
    public int getLastUpdated() {return lastUpdated;}

    @Override
    public void print(Printer printer) {
        // Define how to print this document
        // ...
        printer.sendToQueue(this);
    }

    @Override
    public int compareTo(TextDocument other) {
        return title.compareTo(other.title); // compare the title strings
    }

    // And more TextDocument specific methods...
}
```

Varför interface?

- Interface gör det möjligt att separera defintionen av klassens gränssnitt från implementationen
- Flexibilitet i implementationen
  - Interface passar i situationer där många klasser, som i övrigt saknar gemensamma egenskaper, ska erbjuda (någon) gemensam funktionalitet …
  - … dvs. vi vill erbjuda viss gemensam funktionalitet utan att ärva från en superklass
  - Låg koppling, i detta fall …
  - … andra klasser, “klientkod” kan anropa metoder, via en interface-referens, utan att behöva veta hur metoden är implementerad *eller ens vilken specific implementation (typ/klass) som anropas*